Function definition

def stdNBgraph(dataset):

- def → used to create a function.
- stdNBgraph → function name.
- dataset → input/argument passed to the function.
- : → tells Python that the function body starts here.

we are creating a function called **stdNBgraph()**.

stdNBgraph(dataset["ssc_p"])

the ssc_p column is passed into the function.

Import Seaborn

import seaborn as sns

This imports the **Seaborn** library.

Seaborn is mainly used for **data visualization and statistical graphs**.

We give it the short name: sns

Calculate mean

mean = dataset.mean()

This calculates the **mean (average)** of the dataset.

dataset = [40, 50, 60, 70, 80]

Mean:

(40 + 50 + 60 + 70 + 80) / 5

= 300 / 5

= 60

mean = 60

Calculate standard deviation

std = dataset.std()

This calculates the **standard deviation** of the dataset.

Standard deviation tells us **how much the values are spread around the mean**.

Mean = 60

Values close to 60 → smaller standard deviation.

Values far away from 60 → larger standard deviation.

std = dataset.std()

stores the standard deviation in the variable std.

Create values list

values = [i for i in dataset]

This is called a **list comprehension**.

Take every value from dataset and put it into a new list called values.

dataset = [50, 60, 70, 80]

values = [50, 60, 70, 80]

[i for i in dataset]

There are three important parts:

[i for i in dataset]

↑ ↑ ↑

store loop source

i

i

is the loop variable.

for i

Take each value one by one and temporarily call it i.

in dataset

Get those values from dataset.

values = [i for i in dataset]

is equivalent to:

values = []

for i in dataset:

  values.append(i)

Both produce the same result.

Calculate Z-score

z_score = [((j - mean) / std) for j in values]

This is the **most important line** in this program.

It calculates the **Z-score for every value**.

The Z-score formula is:

$$Z = \frac{X - Mean}{Standard\ Deviation}$$

In Python:

(j - mean) / std

Where:

- j → current data value
- mean → average
- std → standard deviation

Understanding the list comprehension

[((j - mean) / std) for j in values]

Take every value j from values, calculate its Z-score, and store the result in a list.

values = [40, 50, 60, 70, 80]

mean = 60

std = 10

For 40:

(40 - 60) / 10

= -20 / 10

= -2

For 50:

(50 - 60) / 10

= -1

For 60:

(60 - 60) / 10

= 0

For 70:

(70 - 60) / 10

= 1

For 80:

(80 - 60) / 10

= 2

z_score = [-2, -1, 0, 1, 2]

Traditional for loop

z_score = [((j - mean) / std) for j in values]

is equivalent to:

```
z_score = []
for j in values:
    z = (j - mean) / std
    z_score.append(z)
```

Plot the Z-score distribution

sns.distplot(z_score, kde=True)

This creates a distribution plot for the Z-scores.

distplot

Creates a distribution plot.

z_score

This is the data we want to plot.

kde=True

KDE means:

Kernel Density Estimation

It adds a smooth curve representing the distribution of the data.

Plot the distribution of the calculated Z-scores and show the smooth KDE curve.

Calculate average of Z-scores

sum(z_score) / len(z_score)

This calculates the **mean of the Z-scores**.

sum(z_score)

Adds all the Z-scores.

z_score = [-2, -1, 0, 1, 2]

sum(z_score)

0

len(z_score)

Counts how many Z-scores are present.

len(z_score)

5

sum(z_score) / len(z_score)

0 / 5

= 0

The mean of standardized Z-scores should be approximately:

0

Calling the function

stdNBgraph(dataset["ssc_p"])

This calls the function.

dataset["ssc_p"]